

EX N0:4**14/02/2025****SUB QUERIES****AIM:**

To execute Nested Query using IN, NOT IN, Some, ALL, Exists and Not Exists keywords execute scalar sub query in select and insert clause.

A Sub-query or Inner query or a Nested query is a query within another SQL query and embedded within the WHERE clause.

TYPES:

- Independent Sub Query
- Correlated Sub Query
- Scalar Sub Query

TABLE:

ID	NAME	DEPT	SALARY	AGE
-----	-----	----	-----	-----
101	Arun	CSE	50000	35
102	Bala	ECE	60000	40
103	Yokarajan	IT	55000	27
104	Ram	IT	58000	42
105	Raju	EEE	40000	45
106	Dharun	CHEM	35000	24

6 rows selected.

INDEPENDENT SUB QUERY:**SET MEMBERSHIP:****Using “in” keyword:**

```
SQL> select id, name, age from Staff where salary in (select salary from  
Staff where salary > 40000);
```

ID	NAME	AGE
101	Arun	35
102	Bala	40
103	Yokarajan	27
104	Ram	42

Using not in keyword:

SQL> select name, salary from Staff where age not in (select age from Staff where salary > 40000);

NAME	SALARY
Raju	40000
Dharun	35000

SET COMPARISON:

Using some keyword:

Using relational operators:

SQL> select id, name, age from Staff where salary > some (select salary from Staff where salary < 40000);

ID	NAME	AGE
101	Arun	35
102	Bala	40
103	Yokarajan	27
104	Ram	42
105	Raju	45

SQL> select id, name, age from Staff where salary < some (select salary from Staff where salary > 40000);

ID	NAME	AGE
101	Arun	35
103	Yokarajan	27
104	Ram	42
105	Raju	45
106	Dharun	24

SQL> select id, name, age from Staff where salary >= some (select salary from Staff where salary > 50000);

ID	NAME	AGE
102	Bala	40
103	Yokarajan	27
104	Ram	42

SQL> select id, name, age from Staff where salary <= some (select salary from Staff where salary < 40000);

ID	NAME	AGE
106	Dharun	24

SQL> select id, name, age from Staff where salary <> some (select salary from Staff where age = 40);

ID	NAME	AGE
101	Arun	35
103	Yokarajan	27
104	Ram	42
105	Raju	45
106	Dharun	24

Using all keyword:

Using relational operators:

SQL> select name, salary from Staff where salary > all (select salary from Staff where salary < 40000);

NAME	SALARY

Arun	50000
Bala	60000
Yokarajan	55000
Ram	58000
Raju	40000

SQL> select name, salary from Staff where salary < all (select salary from Staff where age < 24);

NAME	SALARY

Arun	50000
Bala	60000
Yokarajan	55000
Ram	58000
Raju	40000
Dharun	35000

SQL> select name, salary from Staff where salary >= all (select salary from Staff where age > 20);

NAME	SALARY

Bala	60000

SQL> select name, salary from Staff where salary <= all (select salary from Staff where salary < 40000);

NAME	SALARY
------	--------

Dharun	35000
--------	-------

```
SQL> select name, salary from Staff where salary <> all (select salary
from Staff where salary < 50000);
```

NAME	SALARY
------	--------

Ram	58000
-----	-------

Bala	60000
------	-------

Arun	50000
------	-------

Yokarajan	55000
-----------	-------

SET CARDINALITY:

Using exists keyword:

```
SQL> select id, name from Staff e where exists (select age from Staff
where age < 30 and e.age = age);
```

ID	NAME
----	------

103	Yokarajan
-----	-----------

106	Dharun
-----	--------

Using not exists keyword:

```
SQL> select name, salary from Staff e where not exists (select age from
Staff where age > 35 and e.age = age);
```

NAME	SALARY
------	--------

Arun	50000
------	-------

Dharun	35000
--------	-------

Yokarajan	55000
-----------	-------

CORRELATED SUB QUERY:

The inner query is executed separately for each row of the outer query.

```
SQL> select e.id, e.name, e.salary, e.age from Staff e where e.salary <
(select avg(salary) from Staff e2 where e2.age > e.age);
```

ID	NAME	SALARY	AGE
101	Arun	50000	35
106	Dharun	35000	24

SCALAR SUBQUERY:

A scalar subquery is a subquery that selects only one column or expression and returns one value or one row.

```
SQL> select * from teacher1;
```

ID	NAME	DEPT	SALARY
101	Arun	CSE	50000
102	Bala	ECE	60000
103	Yokarajan	IT	55000
104	Ram	IT	58000
105	Raju	EEE	40000
106	Dharun	CHEM	35000

6 rows selected.

```
SQL> select * from CourseAllocation;
```

ID	COUR	NOCLASS
103	OS	6
106	ML	5
101	DSA	5
107	CSE	4
105	EM	4
109	SE	3

108 TD 6
7 rows selected.

```
SQL> select age, (select count(*) from CourseAllocation where  
CourseAllocation.id = Staff.id) as no from Staff;
```

AGE	NO
35	1
40	0
27	1
42	0
45	1
24	1

6 rows selected.

SUB QUERIES :

USING INSERT CLAUSE:

```
SQL> create table stats(sum_val number(10), max_val number(10), min_val  
number(10), avg_val number(10,2));
```

Table created.

```
SQL> select * from stats;
```

no rows selected

```
SQL> insert into stats(sum_val, max_val, min_val, avg_val) values ((select  
sum(salary) from staff), (select max(salary) from staff), (select  
min(salary) from staff), (select avg(salary) from staff));
```

1 row created.

```
SQL> select * from stats;
```

SUM_VAL	MAX_VAL	MIN_VAL	AVG_VAL
298000	60000	35000	49666.67

USING FROM CLAUSE:

SQL> select age, min_sal from (select age, min(salary) as min_sal from Staff group by age) where min_sal > 30000;

AGE	MIN_SAL
42	58000
24	35000
35	50000
40	60000
45	40000
27	55000

USING SELECT CLAUSE:

SQL> select name, (select count(*) from CourseAllocation where CourseAllocation.id = Staff.id) as no from Staff;

NAME	NO
Arun	1
Bala	0
Yokarajan	1
Ram	0
Raju	1
Dharun	1

USING TEMPORARY TABLE WITH CLAUSE:

SQL> with demo as (select id, age from staff where age > 30) select staff.id, staff.age from staff, demo where staff.id = demo.id and staff.salary < 50000;

ID	AGE
105	45

TABLE WITH COMPOSITE PRIMARY KEY AND CHECK CONSTRAINT:

```
SQL> create table schedule(id number(3), name varchar2(20), dept
varchar2(10), primary key(id, name), check (dept in('cse', 'ece', 'it')));
```

Table created.

```
SQL> insert into schedule values(&id, '&name', '&dept');
```

Enter value for id: 101

Enter value for name: Arun

Enter value for dept: cse

```
old 1: insert into schedule values(&id, '&name', '&dept')
```

```
new 1: insert into schedule values(101, 'Arun', 'cse')
```

1 row created.

```
SQL> /
```

Enter value for id: 102

Enter value for name: Bala

Enter value for dept: ece

```
old 1: insert into schedule values(&id, '&name', '&dept')
```

```
new 1: insert into schedule values(102, 'Bala', 'ece')
```

1 row created.

```
SQL> /
```

Enter value for id: 103

Enter value for name: Yokarajan

Enter value for dept: it

```
old 1: insert into schedule values(&id, '&name', '&dept')
```

```
new 1: insert into schedule values(103, 'Yokarajan', 'it')
```

1 row created.

SQL> /

Enter value for id: 104

Enter value for name: Ram

Enter value for dept: cse

old 1: insert into schedule values(&id, '&name', '&dept')

new 1: insert into schedule values(104, 'Ram', 'cse')

1 row created.

SQL> /

Enter value for id: 103

Enter value for name: Arun

Enter value for dept: it

old 1: insert into schedule values(&id, '&name', '&dept')

new 1: insert into schedule values(103, 'Arun', 'it')

1 row created.

SQL> /

Enter value for id: 102

Enter value for name: Bala

Enter value for dept: ece

old 1: insert into schedule values(&id, '&name', '&dept')

new 1: insert into schedule values(102, 'Bala', 'ece')

insert into schedule values(102, 'Bala', 'ece')

*

ERROR at line 1:

ORA-00001: unique constraint (SYSTEM.SYS_C004237) violated

SQL> /

Enter value for id: 106

Enter value for name: Fathima

Enter value for dept: ai

```
old 1: insert into schedule values(&id, '&name', '&dept')
```

```
new 1: insert into schedule values(106, 'Fathima', 'ai')
```

```
insert into schedule values(106, 'Fathima', 'ai')
```

```
*
```

ERROR at line 1:

ORA-02290: check constraint (SYSTEM.SYS_C004236) violated

```
SQL> select * from schedule;
```

ID	NAME	DEPT
101	Arun	cse
102	Bala	ece
103	Yokarajan	it
104	Ram	cse
103	Arun	it

MARKS SPLITUP:

CONTENTS	MARKS ALLOTED	MARKS OBTAINED
Aim, Algorithm, SQL, PL/SQL	30	
Execution and Result	20	
Viva	10	
Total	60	

Result:

Thus Sub Queries using IN, NOT IN, Some, ALL, Exists and Not Exists keywords execute scalar sub query in select and insert clause are executed.